

Final Project

Hotel Reservation & Flight Booking System

1. Project Overview

This project covers the design and implementation of a Hotel Reservation and Flight Booking System using PostgreSQL.

It simulates a real-world travel platform that enables users to search, book, and manage hotel stays and flight reservations efficiently.

The system supports key features such as user registration and role management, hotel and room management, flight scheduling, booking processing, payment tracking, and customer reviews.

It is designed for both customer-facing operations (such as searching and booking) and administrative needs (such as reporting and performance analysis).

The database design prioritizes data integrity, scalability, and real-world applicability.

Core features include trigger-based logic to prevent overlapping hotel bookings, enforcement of seat uniqueness in flight bookings, and the use of constraints and indexes to ensure consistency and improve performance.

In addition, the project includes a set of business-focused SQL queries that serve as reports and operational tools.

These queries provide insights into booking trends, customer behavior, hotel performance, and daily operations.

Overall, the project demonstrates how relational database design can be applied to build a functional, efficient, and scalable booking system that supports both transactional operations and business intelligence.

2. System Requirements

2.1 User Management

The system allows users to register and log in securely. Each user record stores the user's name, email, phone number, and password. The system supports multiple roles, primarily customers and administrators, to control access and available functionality.

2.2 Hotel Management

The system stores detailed information about hotels, including the hotel name, location, star rating, and description. Each hotel contains multiple rooms, and each room has its own attributes such as room type, capacity, price per night, and availability status.

2.3 Hotel Booking

Users can search for hotels by location and view available rooms. They can book rooms by selecting check-in and check-out dates. The system automatically calculates the total booking cost and tracks booking status as either confirmed or cancelled. It also prevents overlapping bookings for the same room.

2.4 Flight Management

The system maintains information about airlines and flights.

Each flight includes details such as departure city, arrival city, departure and arrival times, ticket price, and available seats.

2.5 Flight Booking

Users can search for flights based on route. availability.

They can book flights and select seat numbers.

The system tracks booking status and ensures that the same seat cannot be assigned to multiple confirmed bookings on the same flight.

2.6 Payment System

The system supports payment processing for both hotel and flight bookings. Each payment record includes payment amount, payment method, and payment date. Every payment is linked to exactly one booking, ensuring clear financial tracking.

2.7 Reviews & Ratings

Users can provide ratings and written reviews for hotels. Each review is associated with a specific user and hotel. Ratings are restricted to a defined range to maintain consistency and reliability.

2.8 General System Requirements

The system ensures data integrity through the use of primary keys, foreign keys, constraints, and triggers. It supports multiple bookings per user while preventing invalid operations such as overlapping hotel reservations or duplicate seat assignments. Additionally, the system maintains historical data for bookings, payments, and reviews to support reporting and analysis.

3. Database Design

3.1 Entity Identification

The system consists of multiple entities that represent the core components of the booking platform. Each entity corresponds to a table in the database.

The main entities in the system are:

- Roles: Defines different user roles, such as customer and administrator.
- Users: Stores user information, including personal and authentication data.
- Hotels: Contains hotel details, such as name, location, and rating.
- Rooms: Represents individual rooms within hotels, each with its own properties.
- Hotel Bookings: Stores reservations made by users for hotel rooms.
- Airlines: Contains airline information.
- Flights: Stores flight details, including routes, schedules, and pricing.
- Flight Bookings: Stores reservations made by users for flights.
- Payments: Records payment transactions for bookings.
- Hotel Reviews: Stores user feedback and ratings for hotels.

Each of these entities was designed to reflect a real-world object in the booking system.

3.2 Relationships

The relationships between entities define how data is connected within the system.

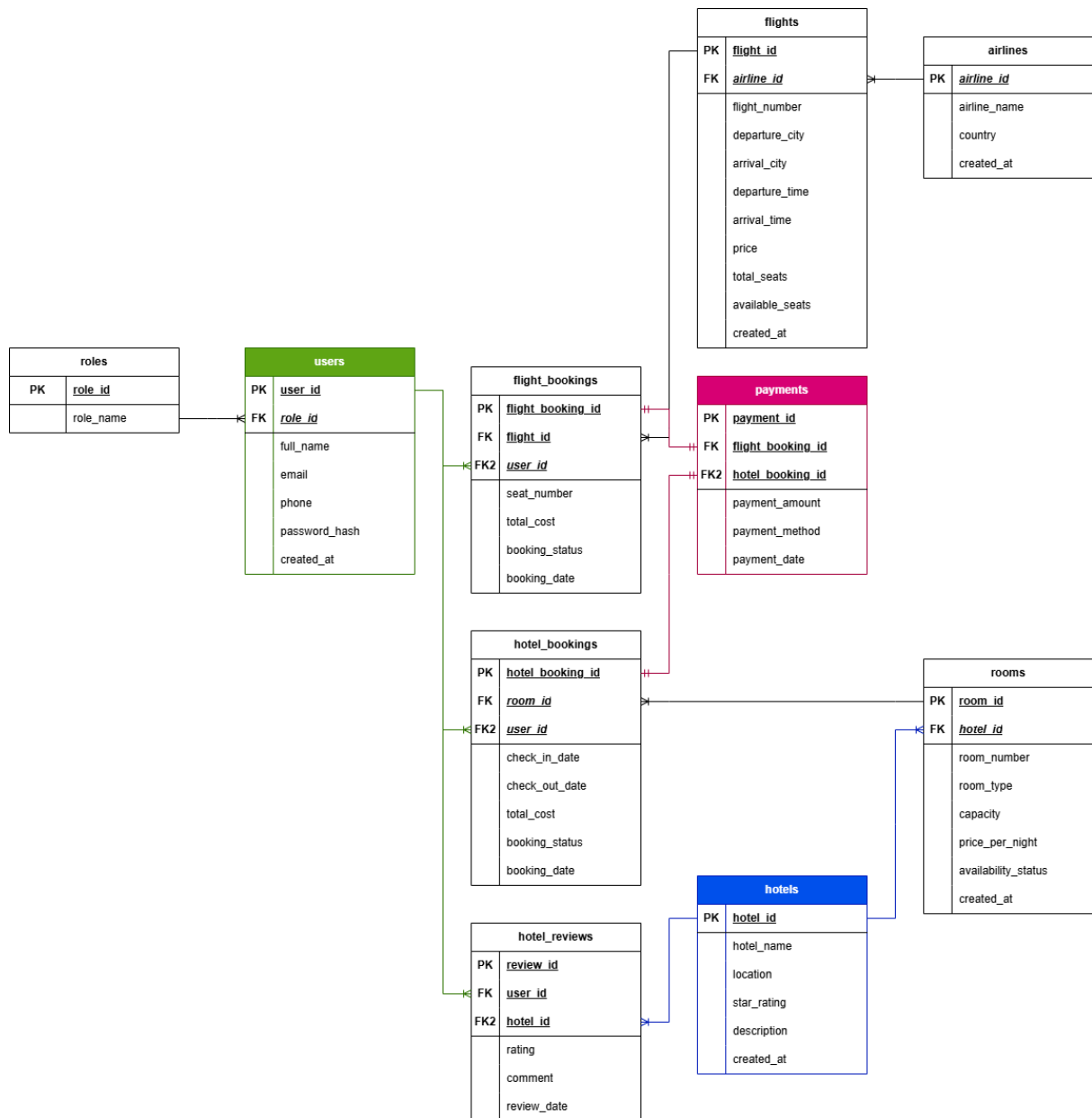
- A role can be assigned to multiple users, while each user has one role (1:M).
- A user can create multiple hotel bookings and flight bookings (1:M).
- A hotel contains multiple rooms, while each room belongs to one hotel (1:M).
- A room can have multiple bookings over time, but each booking belongs to one room (1:M).
- An airline can operate multiple flights, while each flight belongs to one airline (1:M).
- A flight can have multiple bookings, while each booking is associated with one flight (1:M).
- A user can write multiple reviews, and each review belongs to one hotel (1:M).
- Each hotel can have multiple reviews from different users (1:M).
- Each booking (hotel or flight) is associated with exactly one payment (1:1).

Many-to-many relationships (such as users and hotels) are resolved using intermediate tables like `hotel_bookings` and `flight_bookings`.

3.3 ERD (Entity Relationship Diagram)

The Entity Relationship Diagram (ERD) visually represents the database structure, including entities, attributes, and relationships.

The ERD illustrates how all tables are connected through primary and foreign keys, as well as the cardinality of each relationship.



3.4 Design Decisions

Several important design decisions were made to ensure the system is scalable, maintainable, and aligned with real-world requirements.

- **Separate Booking Tables:**
Hotel bookings and flight bookings are stored in separate tables because they have different attributes and business logic. This improves clarity and normalization.
- **Use of Rooms Table:**
Rooms are separated from hotels to allow each hotel to manage multiple room types, capacities, and pricing structures.

- **Payments as a Separate Entity:**
Payments are stored in a separate table to ensure proper financial tracking and to maintain flexibility in linking payments to either hotel or flight bookings.
 - **Reviews Structure:**
Hotel reviews are stored independently to allow users to provide feedback and ratings, which are essential for service evaluation.
 - **Data Integrity:**
Primary keys, foreign keys, and constraints are used throughout the database to ensure consistency and prevent invalid data.
 - **Business Logic Enforcement:**
Triggers are used to enforce rules such as preventing overlapping hotel bookings, ensuring the system behaves correctly even at the database level.
-

4. Database Schema Implementation

4.1 Tables & Constraints

The database schema was implemented using PostgreSQL, with a focus on ensuring data integrity and consistency.

Each table in the system is defined with a primary key to uniquely identify records. Relationships between tables are enforced using foreign keys, which maintain referential integrity.

Several types of constraints were applied:

- **Primary Key Constraints:**
Each table includes a primary key to uniquely identify each record.
- **Foreign Key Constraints:**
Foreign keys are used to link related tables, such as linking users to bookings and rooms to hotels.
- **Unique Constraints:**
Certain fields, such as user email and room number within a hotel, are restricted to unique values to avoid duplication.
- **Check Constraints:**
Check constraints are used to enforce valid data ranges and conditions. For

example:

- Ratings must be between 1 and 5
- Prices must be positive values
- Check-out dates must be later than check-in dates
- A payment must be linked to only one booking type

These constraints ensure that only valid and consistent data is stored in the system.

4.2 Trigger Logic

To enforce business rules at the database level, a trigger was implemented to prevent overlapping hotel bookings.

The trigger is executed before inserting or updating a booking. It checks whether the selected room is already booked for the given time period. If an overlapping confirmed booking exists, the operation is rejected.

This ensures that:

- A room cannot be double-booked
- Booking conflicts are handled automatically by the database

This approach improves reliability by enforcing business logic directly within the database rather than relying only on application-level validation.

Trigger & Function Code:

```
CREATE OR REPLACE FUNCTION prevent_overlapping_hotel_bookings()  
RETURNS TRIGGER  
AS $$  
BEGIN  
    IF NEW.booking_status = 'confirmed' THEN  
        IF EXISTS (  
            SELECT 1  
            FROM hotel_bookings hb  
            WHERE hb.room_id = NEW.room_id  
                  AND hb.booking_status = 'confirmed'  
                  AND hb.hotel_booking_id <> COALESCE(NEW.hotel_booking_id, 0)  
        ) THEN  
            RAISE EXCEPTION 'Room % is already booked for the given time period.', NEW.room_id;  
        END IF;  
    END IF;  
    RETURN NEW;  
END;
```

```

        AND NEW.check_in_date < hb.check_out_date
        AND NEW.check_out_date > hb.check_in_date
    ) THEN
        RAISE EXCEPTION 'This room is already booked fo
r the selected dates.';
    END IF;
END IF;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_prevent_overlapping_hotel_bookings
BEFORE INSERT OR UPDATE
ON hotel_bookings
FOR EACH ROW
EXECUTE FUNCTION prevent_overlapping_hotel_bookings();

```

Trigger Results:

```

Data Output  Messages  Notifications
ERROR:  This room is already booked for the selected dates.
CONTEXT:  PL/pgSQL function prevent_overlapping_hotel_bookings() line 13 at RAISE

SQL state: P0001

```

4.3 Indexes

Indexes were created to improve query performance, especially for frequently searched and joined columns.

The following types of columns were indexed:

- Foreign Keys:
Indexes were added to foreign key columns such as user_id, room_id, and flight_id to speed up joins.
- Search Columns:
Columns frequently used in filtering, such as hotel location and flight routes, were indexed to improve search performance.

- **Booking and Status Columns:**
Indexes were added on booking status and date columns to optimize reporting and operational queries.

These indexes allow the database to retrieve data more efficiently, especially as the dataset grows larger.

5. Sample Data

To test the functionality of the system and ensure that all queries produce meaningful results, a comprehensive set of sample data was inserted into the database.

The dataset was designed to simulate real-world usage and includes:

- Multiple users with different activity levels
- Several hotels located in different cities with varying ratings
- Multiple rooms per hotel with different types and pricing
- Hotel bookings distributed across different dates and months
- Confirmed and cancelled bookings to test booking status logic
- Back-to-back bookings for specific rooms to test operational scenarios
- Flight data across different routes and airlines
- Flight bookings with assigned seat numbers
- Payments linked to both hotel and flight bookings using different payment methods
- Hotel reviews with varied ratings to support performance analysis

The data was intentionally structured to support all types of queries in the system, including customer queries, administrative reports, and operational monitoring queries.

This approach ensures that:

- Queries return meaningful and non-empty results
 - Business scenarios are realistically represented
 - System behavior can be validated under different conditions
-

6. SQL Queries & Business Scenarios

This section presents the SQL queries developed for the system. The queries were designed not only to retrieve data, but also to support realistic business scenarios across three perspectives:

- Customer-side queries, which support searching, filtering, and viewing booking information
- Admin/reporting queries, which support business analysis and decision-making
- Operational queries, which support daily monitoring and system control

Each query is documented using the following structure:

- Scenario
- Business Question
- Purpose
- SQL Query
- Result Screenshot
- Insight

6.1 Customer Queries

Query 1 — Available Hotel Rooms Search

Scenario:

A customer wants to search for hotel rooms in Cairo for a specific travel period and needs to see only rooms that are actually available.

Business Question:

Which rooms are available in a selected city for a given stay period?

Purpose:

Helps customers search for bookable rooms while avoiding overlapping bookings.

```
SELECT
    h.hotel_name AS "Hotel Name",
```

```

        h.location AS "City",
        h.star_rating AS "Hotel Rating",
        r.room_number AS "Room Number",
        r.room_type AS "Room Type",
        r.capacity AS "Capacity",
        r.price_per_night AS "Price Per Night"
FROM hotels h
JOIN rooms r
ON h.hotel_id = r.hotel_id
WHERE h.location = 'Cairo'
AND r.availability_status = 'available'
AND NOT EXISTS (
    SELECT 1
    FROM hotel_bookings hb
    WHERE hb.room_id = r.room_id
    AND hb.booking_status = 'confirmed'
    AND DATE '2026-04-01' < hb.check_out_date
    AND DATE '2026-04-05' > hb.check_in_date
)
ORDER BY h.star_rating DESC, r.price_per_night ASC;

```

Result Screenshot:

	Hotel Name character varying (150)	City character varying (150)	Hotel Rating numeric (2,1)	Room Number character varying (20)	Room Type character varying (50)	Capacity integer	Price Per Night numeric (10,2)
1	Nile View Hotel	Cairo	4.5	101	Single	1	800.00
2	Nile View Hotel	Cairo	4.5	102	Double	2	1200.00
3	Nile View Hotel	Cairo	4.5	103	Suite	4	2500.00
4	Skyline Inn	Cairo	3.8	401	Single	1	700.00
5	Skyline Inn	Cairo	3.8	403	Suite	3	1700.00

Insight:

The system filters out rooms that are already booked within the selected date range, ensuring only valid booking options are displayed.

Query 2 — Filter and Compare Hotel Options

Scenario:

A customer wants to compare hotel rooms based on type, price range, and hotel quality before making a booking decision.

Business Question:

What hotel room options match a selected type and price range, but only in hotels rated above the average rating?

Purpose:

Helps customers compare stronger hotel options with better value and quality.

SQL Query:

```
SELECT
    h.hotel_name AS "Hotel Name",
    h.location AS "City",
    h.star_rating AS "Hotel Rating",
    r.room_type AS "Room Type",
    r.capacity AS "Capacity",
    r.price_per_night AS "Price Per Night"
FROM hotels h
JOIN rooms r
ON h.hotel_id = r.hotel_id
WHERE r.availability_status = 'available'
AND r.room_type = 'Double'
AND r.price_per_night BETWEEN 800 AND 2000
AND h.star_rating > (
    SELECT AVG(h2.star_rating)
    FROM hotels h2
)
ORDER BY h.star_rating DESC, r.price_per_night ASC;
```

Result Screenshot:

	Hotel Name character varying (150) 🔒	City character varying (150) 🔒	Hotel Rating numeric (2,1) 🔒	Room Type character varying (50) 🔒	Capacity integer 🔒	Price Per Night numeric (10,2) 🔒
1	Sea Breeze Hotel	Alexandria	5.0	Double	2	1500.00
2	Palm Garden Hotel	Sharm El Sheikh	4.7	Double	2	1600.00
3	Nile View Hotel	Cairo	4.5	Double	2	1200.00
4	Nile View Hotel	Cairo	4.5	Double	2	1400.00

Insight:

The query narrows results to higher-quality hotels, helping users make more informed and confident booking decisions.

Query 3 — Customer Booking History

Scenario:

A customer or support agent wants to view all bookings made by a specific user, including both hotel and flight bookings.

Business Question:

What is the full booking history of a selected customer across hotel and flight bookings?

Purpose:

Helps customers and support teams review all bookings in one place.

SQL Query:

```
SELECT
    u.full_name AS "Customer Name",
    'Hotel' AS "Booking Type",
    h.hotel_name AS "Booking Name",
    h.location AS "Destination",
    hb.booking_date AS "Booking Date",
    hb.check_in_date AS "Start Date",
    hb.check_out_date AS "End Date",
    hb.total_cost AS "Total Cost",
    hb.booking_status AS "Status"
FROM users u
JOIN hotel_bookings hb
ON u.user_id = hb.user_id
JOIN rooms r
ON hb.room_id = r.room_id
JOIN hotels h
ON r.hotel_id = h.hotel_id
WHERE u.user_id = 1
UNION
SELECT
    u.full_name AS "Customer Name",
    'Flight' AS "Booking Type",
    f.flight_number AS "Booking Name",
    f.arrival_city AS "Destination",
```

```

        fb.booking_date AS "Booking Date",
        f.departure_time::date AS "Start Date",
        f.arrival_time::date AS "End Date",
        fb.total_cost AS "Total Cost",
        fb.booking_status AS "Status"
FROM users u
JOIN flight_bookings fb
ON u.user_id = fb.user_id
JOIN flights f
ON fb.flight_id = f.flight_id
WHERE u.user_id = 1;

```

Result Screenshot:

	Customer Name character varying (150)	Booking Type text	Booking Name character varying	Destination character varying	Booking Date timestamp without time zone	Start Date date	End Date date	Total Cost numeric (12,2)	Status character varying (20)
1	Ahmed Hassan	Flight	MS101	Dubai	2026-03-15 10:00:00	2026-04-01	2026-04-...	5000.00	confirmed
2	Ahmed Hassan	Flight	MS404	Jeddah	2026-03-25 14:00:00	2026-04-05	2026-04-...	4000.00	confirmed
3	Ahmed Hassan	Hotel	Desert Oasis Res...	Giza	2026-06-20 16:50:00	2026-07-10	2026-07-...	3000.00	confirmed
4	Ahmed Hassan	Hotel	Nile View Hotel	Cairo	2026-01-01 10:00:00	2026-01-10	2026-01-...	1600.00	confirmed
5	Ahmed Hassan	Hotel	Nile View Hotel	Cairo	2026-01-25 11:00:00	2026-02-05	2026-02-...	3600.00	confirmed
6	Ahmed Hassan	Hotel	Palm Garden Hotel	Sharm El Sheikh	2026-04-20 12:00:00	2026-05-10	2026-05-...	4800.00	confirmed
7	Ahmed Hassan	Hotel	Sea Breeze Hotel	Alexandria	2026-03-01 09:30:00	2026-03-15	2026-03-...	4500.00	confirmed

Insight:

The system combines hotel and flight bookings into a unified view, giving a complete overview of customer activity.

Query 4 — Booking Details with Payment Information

Scenario:

A customer wants to view detailed information about a specific hotel booking along with its payment details.

Business Question:

What are the details of a selected hotel booking and its payment?

Purpose:

Helps customers and support staff review booking and payment details together.

SQL Query:

```

SELECT
    hb.hotel_booking_id AS "Booking ID",
    u.full_name AS "Customer Name",
    h.hotel_name AS "Hotel Name",
    h.location AS "City",
    r.room_number AS "Room Number",
    r.room_type AS "Room Type",
    hb.check_in_date AS "Check-In Date",
    hb.check_out_date AS "Check-Out Date",
    hb.total_cost AS "Booking Total",
    hb.booking_status AS "Booking Status",
    p.payment_amount AS "Payment Amount",
    p.payment_method AS "Payment Method",
    p.payment_date AS "Payment Date"
FROM hotel_bookings hb
JOIN users u
    ON hb.user_id = u.user_id
JOIN rooms r
    ON hb.room_id = r.room_id
JOIN hotels h
    ON r.hotel_id = h.hotel_id
JOIN payments p
    ON hb.hotel_booking_id = p.hotel_booking_id
WHERE hb.hotel_booking_id = 1;

```

Result Screenshot:

	Customer Name character varying (150)	Hotel Name character varying (150)	City character varying (150)	Room Number character varying (20)	Room Type character varying (50)	Check-In Date date	Check-Out Date date	Booking Total numeric (12,2)	Booking Status character varying (20)	Payment Amount numeric (12,2)	Payment Method character varying (30)	Payment Date timestamp without time zone
1	Ahmed Hassan	Nile View Hotel	Cairo	101	Single	2026-01-10	2026-01-12	1600.00	confirmed	1600.00	credit_card	2026-01-01 10:05:00

Insight:

The query provides a detailed breakdown of booking and payment information, improving transparency and support efficiency.

Query 5 — Flight Search by Route

Scenario:

A customer wants to search for flights between two cities and compare available options.

Business Question:

Which flights are available for a selected route?

Purpose:

Helps customers search for flight options by route, time, and price.

SQL Query:

```
SELECT
    a.airline_name AS "Airline",
    f.flight_number AS "Flight Number",
    f.departure_city AS "Departure City",
    f.arrival_city AS "Arrival City",
    f.departure_time AS "Departure Time",
    f.arrival_time AS "Arrival Time",
    f.price AS "Ticket Price",
    f.available_seats AS "Available Seats"
FROM flights f
JOIN airlines a
    ON f.airline_id = a.airline_id
WHERE f.departure_city = 'Cairo'
    AND f.arrival_city = 'Dubai'
    AND f.available_seats > 0
ORDER BY f.price ASC, f.departure_time ASC;
```

Result Screenshot:

	Airline character varying (150)	Flight Number character varying (30)	Departure City character varying (100)	Arrival City character varying (100)	Departure Time timestamp without time zone	Arrival Time timestamp without time zone	Ticket Price numeric (10,2)	Available Seats integer
1	EgyptAir	MS101	Cairo	Dubai	2026-04-01 08:00:00	2026-04-01 12:00:00	5000.00	140
2	Emirates	EK606	Cairo	Dubai	2026-06-01 16:00:00	2026-06-01 20:00:00	5500.00	25

Insight:

The query allows users to compare flights based on availability, pricing, and schedule, supporting better travel planning decisions.

6.2 Admin / Reporting Queries

Query 6 — Hotel Booking Summary Report

Scenario:

Management wants to understand how many bookings each hotel is receiving to evaluate demand across different locations.

Business Question:

How many confirmed bookings does each hotel have?

Purpose:

Helps management measure hotel demand and identify the most popular hotels.

SQL Query:

```
SELECT
    h.hotel_name AS "Hotel Name",
    h.location AS "City",
    COUNT(hb.hotel_booking_id) AS "Confirmed Bookings"
FROM hotels h
JOIN rooms r
    ON h.hotel_id = r.hotel_id
JOIN hotel_bookings hb
    ON r.room_id = hb.room_id
WHERE hb.booking_status = 'confirmed'
GROUP BY h.hotel_name, h.location
ORDER BY "Confirmed Bookings" DESC, "Hotel Name";
```

Result Screenshot:

	Hotel Name character varying (150) 🔒	City character varying (150) 🔒	Confirmed Bookings bigint 🔒
1	Sea Breeze Hotel	Alexandria	7
2	Nile View Hotel	Cairo	5
3	Desert Oasis Resort	Giza	3
4	Palm Garden Hotel	Sharm El Sheikh	3
5	Royal Lotus Hotel	Luxor	2
6	Skyline Inn	Cairo	1

Insight:

The query highlights which hotels receive the highest number of bookings, helping management allocate resources and plan promotions.

Query 7 — Hotel Profitability Report

Scenario:

Management wants to evaluate which hotels generate the most revenue relative to their size.

Business Question:

Which hotels generate the highest revenue per room?

Purpose:

Helps management compare hotel efficiency and profitability.

SQL Query:

```
SELECT
    x."Hotel Name",
    x."Total Rooms",
    x."Total Revenue",
    x."Revenue Per Room",
    RANK() OVER (ORDER BY x."Revenue Per Room" DESC) AS "Profitability Rank"
FROM (
    SELECT
        h.hotel_name AS "Hotel Name",
        COUNT(DISTINCT r.room_id) AS "Total Rooms",
        SUM(hb.total_cost) AS "Total Revenue",
        SUM(hb.total_cost) / COUNT(DISTINCT r.room_id) AS
"Revenue Per Room"
    FROM hotels h
    JOIN rooms r
        ON h.hotel_id = r.hotel_id
    JOIN hotel_bookings hb
        ON r.room_id = hb.room_id
    WHERE hb.booking_status = 'confirmed'
    GROUP BY h.hotel_name
```

```
) x
ORDER BY "Profitability Rank", "Hotel Name";
```

Result Screenshot:

	Hotel Name character varying (150)	Total Rooms bigint	Total Revenue numeric	Revenue Per Room numeric	Profitability Rank bigint
1	Sea Breeze Hotel	2	34500.00	17250.00000000000000	1
2	Palm Garden Hotel	2	19200.00	9600.0000000000000000	2
3	Nile View Hotel	4	19400.00	4850.0000000000000000	3
4	Desert Oasis Resort	2	7200.00	3600.0000000000000000	4
5	Royal Lotus Hotel	2	6600.00	3300.0000000000000000	5
6	Skyline Inn	1	2850.00	2850.0000000000000000	6

Insight:

The query shows which hotels are performing best financially, allowing management to identify high-performing and underperforming properties.

Query 8 — Top Customers by Hotel Spending

Scenario:

The business wants to identify its most valuable customers based on spending behavior.

Business Question:

Which customers spend the most on confirmed hotel bookings?

Purpose:

Helps identify high-value customers for loyalty programs and targeted marketing.

SQL Query:

```
SELECT
    x."Customer Name",
    x."Email",
    x."Phone",
    x."Confirmed Hotel Bookings",
    x."Total Hotel Spending",
    x."Average Spend per Booking",
```

```

RANK() OVER (ORDER BY x."Total Hotel Spending" DESC) AS
"Customer Rank"
FROM (
    SELECT
        u.full_name AS "Customer Name",
        u.email AS "Email",
        u.phone AS "Phone",
        COUNT(hb.hotel_booking_id) AS "Confirmed Hotel Book
ings",
        SUM(hb.total_cost) AS "Total Hotel Spending",
        AVG(hb.total_cost) AS "Average Spend per Booking"
    FROM users u
    JOIN hotel_bookings hb
        ON u.user_id = hb.user_id
    WHERE hb.booking_status = 'confirmed'
    GROUP BY u.full_name, u.email, u.phone
) x
ORDER BY "Customer Rank", "Customer Name";

```

Result Screenshot:

	Customer Name character varying (150)	Email character varying (150)	Phone character varying (30)	Confirmed Hotel Bookings bigint	Total Hotel Spending numeric	Average Spend per Booking numeric	Customer Rank bigint
1	Sara Mostafa	sara@example.com	01044444444	3	24600.00	8200.000000000000000000	1
2	Ahmed Hassan	ahmed@example.com	01011111111	5	17500.00	3500.000000000000000000	2
3	Omar Khaled	omar@example.com	01033333333	4	15000.00	3750.000000000000000000	3
4	Karim Samir	karim@example.com	01077777777	3	13100.00	4366.666666666666666667	4
5	Laila Nabil	laila@example.com	01066666666	2	9800.00	4900.000000000000000000	5
6	Mona Ali	mona@example.com	01022222222	3	7050.00	2350.000000000000000000	6
7	Youssef Adel	youssef@example.com	01055555555	1	2700.00	2700.000000000000000000	7

Insight:

The query ranks customers based on spending, helping the business focus on retaining its most profitable users.

Query 9 — Monthly Booking Trend Report

Scenario:

Management wants to analyze booking activity over time to understand seasonal trends.

Business Question:

How many confirmed hotel bookings were made in each month?

Purpose:

Helps track booking trends and identify peak and low seasons.

SQL Query:

```
SELECT
    TRIM(TO_CHAR(booking_date, 'Month')) AS "Month Name",
    COUNT(hotel_booking_id) AS "Confirmed Hotel Bookings",
    SUM(total_cost) AS "Total Hotel Revenue",
    AVG(total_cost) AS "Average Booking Value"
FROM hotel_bookings
WHERE booking_status = 'confirmed'
GROUP BY TO_CHAR(booking_date, 'Month')
ORDER BY MIN(booking_date);
```

Result Screenshot:

	Month Name text	Confirmed Hotel Bookings bigint	Total Hotel Revenue numeric	Average Booking Value numeric
1	January	2	5200.00	2600.000000000000000000
2	February	3	8100.00	2700.000000000000000000
3	March	5	26550.00	5310.000000000000000000
4	April	3	13800.00	4600.000000000000000000
5	May	4	15700.00	3925.000000000000000000
6	June	3	10800.00	3600.000000000000000000
7	July	1	9600.00	9600.000000000000000000

Insight:

The query reveals monthly trends in bookings and revenue, helping management plan pricing and promotions accordingly.

Query 10 — Revenue by Payment Method Report

Scenario:

The business wants to understand which payment methods are most commonly used and generate the most revenue.

Business Question:

Which payment methods bring the highest revenue?

Purpose:

Helps management understand customer payment behavior and optimize payment options.

SQL Query:

```
SELECT
    payment_method AS "Payment Method",
    COUNT(payment_id) AS "Number of Payments",
    SUM(payment_amount) AS "Total Revenue",
    AVG(payment_amount) AS "Average Payment Amount"
FROM payments
GROUP BY payment_method
ORDER BY "Total Revenue" DESC, "Payment Method";
```

Result Screenshot:

	Payment Method character varying (30) 🔒	Number of Payments bigint 🔒	Total Revenue numeric 🔒	Average Payment Amount numeric 🔒
1	credit_card	11	54050.00	4913.63636363636364
2	wallet	5	23900.00	4780.0000000000000000
3	cash	5	20000.00	4000.0000000000000000
4	bank_transfer	4	19600.00	4900.0000000000000000
5	debit_card	4	11600.00	2900.0000000000000000

Insight:

The query shows which payment methods are most popular and profitable, helping guide financial and operational decisions.

6.3 Operational Queries

Query 11 — Today's Hotel Check-Ins

Scenario:

Hotel front desk staff need to know which guests are arriving today in each hotel to prepare rooms and manage check-ins.

Business Question:

Which guests are checking in today at each hotel?

Purpose:

Helps hotel operations teams prepare for arrivals and manage daily workload efficiently.

SQL Query:

```
SELECT
    h.hotel_name AS "Hotel Name",
    COUNT(hb.hotel_booking_id) AS "Number of Arrivals",
    ARRAY_AGG(u.full_name) AS "Arriving Guests"
FROM hotel_bookings hb
JOIN users u
    ON hb.user_id = u.user_id
JOIN rooms r
    ON hb.room_id = r.room_id
JOIN hotels h
    ON r.hotel_id = h.hotel_id
WHERE hb.check_in_date = CURRENT_DATE
    AND hb.booking_status = 'confirmed'
GROUP BY h.hotel_name
ORDER BY h.hotel_name;
```

Result Screenshot:

	Hotel Name character varying (150) 🔒	Number of Arrivals bigint 🔒	Arriving Guests character varying[] 🔒
1	Sea Breeze Hotel	1	{"Sara Mostafa"}

Insight:

The query provides a clear overview of daily arrivals, helping staff organize check-in operations and improve customer service.

Query 12 — Rooms with High Booking Frequency

Scenario:

Operations want to identify which rooms are used more frequently than others.

Business Question:

Which rooms are booked more often than the average room booking count?

Purpose:

Helps identify heavily used rooms for maintenance planning and pricing strategies.

SQL Query:

```
SELECT
    h.hotel_name AS "Hotel Name",
    r.room_number AS "Room Number",
    r.room_type AS "Room Type",
    COUNT(hb.hotel_booking_id) AS "Confirmed Bookings"
FROM rooms r
JOIN hotels h
    ON r.hotel_id = h.hotel_id
JOIN hotel_bookings hb
    ON r.room_id = hb.room_id
WHERE hb.booking_status = 'confirmed'
GROUP BY h.hotel_name, r.room_number, r.room_type
HAVING COUNT(hb.hotel_booking_id) > (
    SELECT AVG(room_booking_count)
    FROM (
        SELECT COUNT(hb2.hotel_booking_id) AS room_booking_
count
        FROM rooms r2
        JOIN hotel_bookings hb2
            ON r2.room_id = hb2.room_id
        WHERE hb2.booking_status = 'confirmed'
        GROUP BY r2.room_id
    ) sub
)
ORDER BY "Confirmed Bookings" DESC, "Hotel Name";
```

Result Screenshot:

	Hotel Name character varying (150) 🔒	Room Number character varying (20) 🔒	Room Type character varying (50) 🔒	Confirmed Bookings bigint 🔒
1	Sea Breeze Hotel	301	Double	5
2	Desert Oasis Resort	202	Double	2
3	Nile View Hotel	103	Suite	2
4	Palm Garden Hotel	501	Double	2
5	Sea Breeze Hotel	302	Suite	2

Insight:

The query highlights rooms with high demand, helping optimize pricing and maintenance schedules.

Query 13 — Customers with High Cancellation Behavior

Scenario:

The business wants to monitor customers who frequently cancel bookings.

Business Question:

Which customers have more than one cancelled hotel booking?

Purpose:

Helps identify risky booking behavior and improve cancellation policies.

SQL Query:

```
SELECT
    u.full_name AS "Customer Name",
    u.email AS "Email",
    u.phone AS "Phone",
    COUNT(hb.hotel_booking_id) AS "Cancelled Bookings"
FROM users u
JOIN hotel_bookings hb
    ON u.user_id = hb.user_id
WHERE hb.booking_status = 'cancelled'
GROUP BY u.full_name, u.email, u.phone
HAVING COUNT(hb.hotel_booking_id) > 1
ORDER BY "Cancelled Bookings" DESC, "Customer Name";
```

Result Screenshot:

	Customer Name character varying (150) 🔒	Email character varying (150) 🔒	Phone character varying (30) 🔒	Cancelled Bookings bigint 🔒
1	Youssef Adel	youssef@example.com	01055555555	2

Insight:

The query identifies customers with repeated cancellations, allowing the business to take preventive actions.

Query 14 — Hotels with Low Ratings

Scenario:

Management wants to identify hotels that are receiving poor customer feedback.

Business Question:

Which hotels have average guest ratings below a certain threshold?

Purpose:

Helps management detect underperforming hotels and improve service quality.

SQL Query:

```
SELECT
    h.hotel_name AS "Hotel Name",
    h.location AS "City",
    AVG(hr.rating) AS "Average Guest Rating",
    COUNT(hr.review_id) AS "Number of Reviews"
FROM hotels h
JOIN hotel_reviews hr
    ON h.hotel_id = hr.hotel_id
GROUP BY h.hotel_name, h.location
HAVING AVG(hr.rating) < 4.5
ORDER BY "Average Guest Rating" ASC, "Hotel Name";
```

Result Screenshot:

	Hotel Name character varying (150) 🔒	City character varying (150) 🔒	Average Guest Rating numeric 🔒	Number of Reviews bigint 🔒
1	Royal Lotus Hotel	Luxor	3.5000000000000000	2
2	Skyline Inn	Cairo	3.5000000000000000	2
3	Desert Oasis Resort	Giza	4.0000000000000000	1

Insight:

The query highlights hotels with low ratings, helping prioritize improvements and maintain service standards.

Query 15 — Back-to-Back Room Bookings with No Buffer

Scenario:

Hotel operations want to identify rooms that are booked continuously without any gap between reservations.

Business Question:

Which rooms have one booking ending on the same date another booking starts?

Purpose:

Helps detect rooms under operational pressure and manage cleaning or preparation time.

SQL Query:

```
SELECT
    h.hotel_name AS "Hotel Name",
    r.room_number AS "Room Number",
    r.room_type AS "Room Type",
    hb1.hotel_booking_id AS "Current Booking ID",
    hb1.check_out_date AS "Current Check-Out",
    hb2.hotel_booking_id AS "Next Booking ID",
    hb2.check_in_date AS "Next Check-In"
FROM hotel_bookings hb1
JOIN hotel_bookings hb2
    ON hb1.room_id = hb2.room_id
JOIN rooms r
    ON hb1.room_id = r.room_id
```

```

JOIN hotels h
  ON r.hotel_id = h.hotel_id
WHERE hb1.hotel_booking_id <> hb2.hotel_booking_id
  AND hb1.booking_status = 'confirmed'
  AND hb2.booking_status = 'confirmed'
  AND hb1.check_out_date = hb2.check_in_date
ORDER BY h.hotel_name, r.room_number, hb1.check_out_date;

```

Result Screenshot:

	Hotel Name character varying (150)	Room Number character varying (20)	Room Type character varying (50)	Current Booking ID integer	Current Check-Out date	Next Booking ID integer	Next Check-In date
1	Nile View Hotel	103	Suite	16	2026-06-03	17	2026-06-03

Insight:

The query identifies rooms with no buffer between bookings, helping operations teams plan turnover and avoid delays.

The system provides value across multiple levels, supporting both customers and business operations.

7.1 Customer Value

The system allows customers to easily search and book hotels and flights based on their preferences. Features such as filtering by location, price, and room type help users find suitable options quickly. Additionally, customers can view their booking history and payment details, ensuring transparency and a smooth user experience.

7.2 Administrative Value

The system supports management decision-making through various reporting queries. It enables administrators to analyze hotel performance, track revenue, identify top customers, and monitor booking trends over time. These insights help improve pricing strategies, marketing efforts, and overall business performance.

7.3 Operational Value

The system also supports daily operational activities. Queries such as today's check-ins help staff prepare for arrivals, while others identify high-demand

rooms and customers with frequent cancellations. This allows the business to manage resources efficiently, reduce risks, and improve service quality.

Overall, the system combines transactional functionality with analytical capabilities, making it useful for both operational execution and strategic planning.

8. Challenges & Solutions

During the development of the system, several challenges were encountered. Each challenge required a practical solution to ensure the system functions correctly and efficiently.

8.1 Preventing Overlapping Hotel Bookings

Challenge:

Hotel rooms can be booked for specific date ranges, which creates a risk of overlapping bookings for the same room.

Solution:

A database trigger was implemented to check for overlapping date ranges before inserting or updating a booking. If an overlap is detected, the operation is rejected. This ensures that no room can be double-booked.

8.2 Preventing Duplicate Flight Seat Assignments

Challenge:

In flight bookings, the same seat should not be assigned to multiple passengers on the same flight.

Solution:

A unique partial index was created on the combination of flight ID and seat number for confirmed bookings. This ensures that each seat is assigned only once per flight while still allowing cancelled bookings to remain stored.

8.3 Linking Payments to Multiple Booking Types

Challenge:

Payments need to be associated with both hotel bookings and flight bookings, which have different structures.

Solution:

A flexible design was implemented where the payments table includes two foreign keys, one for hotel bookings and one for flight bookings. A check constraint ensures that each payment is linked to only one type of booking.

8.4 Designing Business-Oriented Queries

Challenge:

Initial queries were simple and focused only on retrieving data, without reflecting real business needs.

Solution:

The queries were redesigned to represent real-world scenarios, such as customer searches, management reports, and operational monitoring. Each query was structured with a clear purpose and business benefit.

8.5 Ensuring Meaningful Query Results

Challenge:

Some queries initially returned empty or insignificant results due to limited sample data.

Solution:

The dataset was expanded and diversified to include multiple users, bookings, cancellations, and time ranges. This ensured that all queries produce meaningful and realistic results.

9. Conclusion

This project presented the design and implementation of a Hotel Reservation and Flight Booking System using a relational database approach. The system successfully integrates user management, hotel and flight booking processes, payment tracking, and customer reviews into a unified structure.

Through the use of proper database design principles, constraints, triggers, and indexes, the system ensures data integrity, consistency, and performance. Additionally, the development of business-oriented queries demonstrates how the database can be used not only for data storage but also for analysis and decision-making.

The project highlights the importance of combining technical implementation with real-world business scenarios. It shows how a well-designed database can support both operational activities and strategic insights.

In the future, the system can be extended by integrating a user interface, adding real-time availability updates, and enhancing analytics capabilities to further support business growth.
